
CONWAY’S 99-GRAPH: PARTIAL FORMALIZATION, SYMMETRY-RESTRICTED SEARCH, AND VERIFICATION LIMITS

TECHNICAL NOTE

Antonio Machuca

Universidad Rey Juan Carlos
Madrid, Spain

am.machuca.2023@alumnos.urjc.es

Permanent contact: contactoantoniomachuca@gmail.com

September 21, 2026

ABSTRACT

Conway’s 99-graph problem asks whether a strongly regular graph with parameters $(99, 14, 1, 2)$ exists. This technical note inventories a repository containing a partial Lean 4 formalization, symmetry-restricted SAT encoders, structural calculations, and historical solver records. It corrects earlier claims about novelty and certification. In particular, the restriction that an involution fixes exactly one vertex is prior work of Makhnev–Minakova, explicitly recalled by Makhnev in 2009, and the exclusion of order 7 predates this project. The repository contains checked candidate-verification and structural lemmas, but several advertised graph-level exclusions assume additional arithmetic or group-theoretic premises, and an aggregate classification depends on `sorryAx`. Retained logs record checked refutations of specific CNF formulas; they do not certify the graph-to-CNF translation. A reproduced variable/constant collision in the initially audited canonical order-7 encoder illustrates this distinction despite a passing regression suite. Incidence and stabilizer calculations for the one-fixed-vertex case are retained as reproducible exploratory work. No new graph-theoretic restriction, complete rigidity proof, or resolution of existence is claimed.

Keywords Conway 99-graph · strongly regular graphs · partial formalization · Lean 4 · SAT encoding · proof checking · reproducibility

1 Scope, prior work, and corrected attribution

A strongly regular graph with parameters (v, k, λ, μ) is a k -regular graph on v vertices in which adjacent pairs have λ common neighbors and distinct nonadjacent pairs have μ common neighbors. The existence of an $\text{srg}(99, 14, 1, 2)$ remains open; Conway’s problem list is one widely circulated formulation [1, 2].

For its adjacency matrix A ,

$$A = A^T, \quad \text{diag}(A) = 0, \quad A \in \{0, 1\}^{99 \times 99}, \quad A^2 + A = 12I + 2J. \quad (1.1)$$

The spectrum is

$$\text{Spec}(A) = \{14^1, 3^{54}, (-4)^{44}\}. \quad (1.2)$$

Each neighborhood induces seven disjoint edges, $7K_2$, and the second subconstituent has 84 vertices. These are standard consequences of the parameters, not new results of this project [3, 2].

1.1 The one-fixed-vertex restriction is prior work

Makhnev’s September 2009 lecture slides [4], Theorem 1, explicitly attribute to Makhnev–Minakova [5] a classification of prime-order automorphisms of a hypothetical $(99, 14, 1, 2)$ graph. In its order-2 case, the fixed subgraph is a single

vertex. The preceding discussion lists possible involution fixed subgraphs and eliminates all but this case through character integrality.

Consequently, the assertion that an involution must have $f = 1$ fixed vertex is not a discovery of this repository. Reconstructing exclusions for $f = 3, 5, 7, \dots$ may support formalization or implementation work, but does not establish a previously unknown restriction. A distinct contribution through a new proof, verified implementation, or computational method would require a separate comparison with prior work. Such originality is **PENDING**, not asserted here.

1.2 Other symmetry restrictions and reproduction targets

Behbahani and Lam [6] restricted prime-order automorphisms to orders 2 and 3, a result explicitly recalled in the introduction of Cesarz and Woldar [2]. In particular, order 7 was already excluded before this project.

Crnković and Maksimović [7], Theorem 7.3, exclude groups of orders 6 and 9 and require an order-3 automorphism to have no fixed points. This is not an exclusion of the fixed-point-free order-3 action. The repository's fixed-3 and order-7 models should therefore be described as reproduction experiments for previously excluded cases, not as newly open mathematical branches.

Cesarz and Woldar [2] give computer-free proofs of

$$2 \mid |\text{Aut}(G)| \implies |\text{Aut}(G)| \mid 6, \quad 7 \mid |\text{Aut}(G)| \implies \text{Aut}(G) \cong \mathbb{Z}_7. \quad (1.3)$$

Combining the even-order implication with the exclusion of order 6 leaves order 2. This consequence of prior work is not a new parity-rigidity discovery here. The distinction between earlier computational exclusions and later computer-free proofs is essential.

A recent constraint-programming preprint [8] is relevant methodological background, but a reported timeout does not make a previously excluded symmetry case mathematically open. No controlled runtime comparison or speedup relative to that work is established in this note.

1.3 What this revision does and does not certify

The initial September 21, 2026 documentation audit inspected source and retained logs, ran the Lean build, the then-existing tests, targeted axiom checks, an encoding counterexample, and the incidence/stabilizer calculation. That initial audit did not rerun historical DRAT checks, launch searches, inspect remote processes, or repair encoders.

Subsequent implementation follow-up, September 21. The canonical and legacy Z3-FPF helper defects were subsequently repaired. Selected suites now pass 39 tests: 20 compiler-helper tests, 11 local input-audit tests, and 8 bounded-runner tests. A new audit compares 2,394 cardinality expressions symbolically with adjacency equations in the coordinate model and reproduces the local historical CNF hashes. It also detects an indexing error: historical C selects O_{10} , with intersecting support $\{1, 6\}$, whereas a disjoint representative is O_{11} , with support $\{2, 3\}$. Corrected representatives are A=21, B=1, C=11. Old inputs remain preserved. One bounded local run of corrected C was initiated with user authorization; its status is **EXPLORED**, not a mathematical conclusion. These changes are **COMPILED** implementation work, not new kernel-certified graph exclusions.

Earlier versions claimed unconditional end-to-end formal certification of graph-level exclusions that are actually conditional in the code. For that claim, the required correction is explicit: *The claim is false according to the current state of the repository*. This concerns the scope of the local certification, not the truth of the cited prior mathematical results.

2 Operational taxonomy and formalization boundaries

The repository uses four operational states. The label applies to the precise statement or artifact being checked, not automatically to its intended application.

PROVED. An audited Lean proposition with no `sorryAx` and only the permitted standard foundations; or a recorded formula refutation supported by terminal solver and independent checker verdicts. The formula-level label does not certify its graph encoding.

COMPILED. A successful build or passing limited test suite, including graph-level wrappers that require externally supplied mathematical premises and declarations that depend on `sorryAx`.

EXPLORED. Uncertified Python/SMT calculations, structural computations, and inconclusive searches. Analytical reconstructions not connected to a complete kernel proof are presented at this level in this note.

PENDING. Missing formal interfaces, encoding repairs and coverage audits, originality claims, and unresolved mathematical cases.

Published antecedents are cited as literature, not represented as new local kernel certificates.

2.1 PROVED: candidate checking and selected structural lemmas

The Lean library [9] defines `ConwayAdj` using (1.1). Its Boolean checker tests zero diagonal, symmetry, binary entries, and the matrix equation. The declarations `conway_soundness` and `conway_complete` establish

$$\text{checkConway}(A) = \text{true} \iff \text{ConwayAdj}(A). \quad (2.1)$$

This is a candidate-verification component, not a construction of a candidate graph.

Targeted `#print axioms` checks returned only `[proptext, Quot.sound]` for these two declarations and for:

- `conway_k4_free;`
- `conway_neighborhood_one_factor;`
- `conway_diameter_at_most_two;`
- `conway_z2_f3_fixed_points_dichotomy.`

These are genuine checked consequences of their stated graph hypotheses. Neither novelty nor a complete audit of every declaration in the imported modules follows from this selected inventory.

2.2 PROVED arithmetic does not supply the missing graph hypotheses

The final advertised $f = 5$ exclusion includes, inside its existential premise, a natural number ε_1 satisfying

$$(\varepsilon_1 = 18 \vee \varepsilon_1 = 15) \wedge \varepsilon_1 \bmod 7 = 6. \quad (2.2)$$

The corresponding $f = 7$ declaration assumes

$$\varepsilon_1 \in \{7, 10, 13\} \wedge \varepsilon_1 \bmod 7 = 2. \quad (2.3)$$

Lean correctly refutes these conjunctions. The declarations do not derive the counting identity, spectral congruence, and exhaustive candidate conditions from a graph and its automorphism. The arithmetic incompatibilities are **PROVED**; their advertised complete graph-level applications are **COMPILED**, with the missing interfaces **PENDING**.

Likewise, `ConwayAutGroupBounds` assumes the external group-order restrictions. The parity result deduces a numerical consequence from those fields; it does not construct them for every Conway graph. The Cesarz–Woldar module checks terminal contradictions such as $7a = 62$ or a sum of even numbers equaling 5. It does not formalize the entire graph-action and orbit-classification argument of the published paper.

Inspecting axioms is therefore necessary but insufficient: the logical statement must also be inspected. An unproved application premise can be an ordinary hypothesis without appearing as `sorryAx`.

2.3 COMPILED: unfinished nonexistence declarations

The build completed with 32 jobs, including warnings for `sorry` in the order-7, fixed-3, and fixed-point-free order-3 nonexistence modules. The aggregate declaration `conway_automorphism_group_restricted` reports `[sorryAx, Quot.sound]`.

Its actual statement combines exclusions for specified actions; it is not itself a cardinality classification of the full automorphism group. Neither successful compilation nor an ambitious theorem name establishes $|\text{Aut}(G)| \in \{1, 2\}$.

3 Analytical reconstructions and their scope

Status: EXPLORED. The following calculations explain the constraints and arithmetic checks. They are not presented as new graph-theoretic results or as completed kernel formalizations. Their use in an end-to-end certified graph reduction remains **PENDING**.

3.1 Trace congruence for an involution

Let t be an involution, let $f = |\text{Fix}(t)|$, and let P_t be its permutation matrix. Let a, c denote the $+1$ multiplicities of P_t on the adjacency eigenspaces for 3 and -4 . Since $AP_t = P_tA$ and the all-ones vector is fixed,

$$a + c = \frac{97 + f}{2}. \quad (3.1)$$

If ε_1 is the number of unordered adjacent transposed pairs, then

$$2\varepsilon_1 = \text{Tr}(AP_t) = 14 + 3(2a - 54) - 4(2c - 44) \quad (3.2)$$

$$= 28 + 6a - 8c = 14a - 360 - 4f. \quad (3.3)$$

Hence

$$\varepsilon_1 = 7a - 180 - 2f, \quad \varepsilon_1 \equiv 5(f - 1) \pmod{7}. \quad (3.4)$$

This agrees with Makhnev's character expression [4], which becomes

$$\chi_2(t) = \frac{4f + 2\varepsilon_1 - 18}{7} = 2a - 54. \quad (3.5)$$

The character-integrality obstruction is an antecedent, not a new spectral method introduced here.

3.2 Counting fixed neighbors

Write $F = \text{Fix}(t)$, $U = V(G) \setminus F$, $H = G[F]$, and $m = |E(H)|$. For $u \in U$, put $d(u) = |N(u) \cap F|$. The common-neighbor set of $u, t(u)$ is invariant under t , and its fixed points are exactly the fixed neighbors of u . Thus $d(u) = 1$ if $u \sim t(u)$, while $d(u) \in \{0, 2\}$ otherwise.

Let N_2 count the vertices of U with $d(u) = 2$, and put

$$S = \sum_{z \in F} \binom{\deg_H(z)}{2}. \quad (3.6)$$

Counting edges between F and U gives

$$2\varepsilon_1 + 2N_2 = 14f - 2m. \quad (3.7)$$

Summing common-neighbor counts over unordered pairs of fixed vertices gives

$$N_2 = f(f - 1) - m - S. \quad (3.8)$$

Consequently, with subtraction in the integers,

$$\varepsilon_1 = f(8 - f) + S. \quad (3.9)$$

A negative base term $f(8 - f)$ is not by itself a contradiction: the positive contribution S must also be controlled. Nor does $\lambda = 1$ alone force $\deg_H \leq 2$; the 3×3 lattice graph already has degree 4 and $\lambda = 1$. The previous manuscript's blanket degree bound and consequent classification into paths and long cycles are withdrawn.

If H is a coclique, then $S = 0$ and $\varepsilon_1 \geq 0$ forces $f \leq 8$. Combining (3.4) and (3.9) yields $(f - 1)(f - 2) \equiv 0 \pmod{7}$. The positive odd possibilities up to 8 leave $f = 1$. This reconstructs part of a known exclusion. It does not establish originality or certify the separate higher- f Python/SMT searches, which remain **EXPLORED** under the repository rules.

4 Recorded SAT refutations and a current encoding defect

4.1 PROVED at the recorded formula level only

The retained solver/checker logs contain the terminal records in Table 1. These were inspected in the audit; the DRAT checks were not rerun.

The corresponding log stems are `cadical_z2_f3_case_a`, `cadical_z2_f3_case_b`, `cadical_z7`, and their `drat_trim` counterparts. The order-7 checker reports an input with 176,613 variables and 421,562 clauses, and binary-mode checking of 192,218,491 proof bytes. The input named `conway_z7_canonical.cnf` is not present at the audited repository root, although the proof trace and logs are present. Rechecking requires recovering the exact input and establishing its provenance; generating a new CNF does not establish identity with the historical one.

Table 1: Retained terminal records, not a certificate of graph-encoding correctness.

Input label	Solver record	Checker record
$f = 3$, Case A	s UNSATISFIABLE; 1,040 conflicts	s VERIFIED; 83 clauses in core
$f = 3$, Case B	s UNSATISFIABLE; 1,150 conflicts	s VERIFIED; 259 clauses in core
Canonical order 7	s UNSATISFIABLE; 771.59 s process time	s VERIFIED; 844.008 s verification time

The retained $f = 3$ logs support one Case A record and one Case B record. They do not individually substantiate every partition and conflict count claimed by earlier manuscript tables. The additional order-7 hunter’s terminal UNSAT is corroborating solver output, not another independently checked DRAT proof.

DRAT checking [10] establishes a refutation of the supplied CNF. To infer graph nonexistence, one also needs the implication

$$\text{graph satisfying the branch hypotheses} \implies \text{satisfying assignment of the CNF.} \quad (4.1)$$

This requires correct constraints, justified symmetry cuts, and exhaustive branch coverage. It is not certified by the DRAT trace. Conversely, a SAT model must be decoded and checked against the graph parameters before it constitutes a graph witness.

4.2 COMPILED: passing tests with a reproduced semantic failure

At the initial audit, 13 tests in `tests/test_canonical_sat_compilers.py` passed despite a defect: the unpatched `scripts/build_z7_canonical_cnf.py` allocated primary variable ID 1 and also interpreted integer 1 as true in its conjunction and cardinality helpers.

The following small reproduction was run on that unpatched revision and required no graph search:

```
from scripts.build_z7_canonical_cnf import ConwayZ7CanonicalCompiler

compiler = ConwayZ7CanonicalCompiler(enable_lex=False)
assert compiler.var_map[(0, 0, 1)] == 1
compiler.add_card_equals([1, 2], 1)
print(compiler.cnf.clauses)
```

The unpatched code printed `[-2]`, representing only $\neg x_2$, not $x_1 + x_2 = 1$. It incorrectly accepted $(0, 0)$ and rejected $(0, 1)$. Likewise, `get_and_lit(1, 2)` returned 2 rather than a conjunction of two variable literals. The repaired helper now encodes the exactly-one constraint correctly in this regression example.

The unpatched canonical order-3 helpers and the separate legacy `build_z3_fpf_cnf.py` used the same convention. They have subsequently been repaired and regression-tested. Effective impact on historical inputs still requires model-specific provenance checks; the separate $f = 1$ compiler did not use the constant-1 shortcut.

The defect concerned encoders, not the correctness of the historical DRAT checker. Existing CNFs do not change when source is repaired. A regenerated legacy Z3-FPF input was preserved in a new directory without launching it. Remaining **PENDING** work includes complete formal graph-to-encoding interfaces and verification of any eventual terminal search result. Passing the added tests is classified as **COMPILED**, not as a kernel proof of the full translation.

5 Incidence structure and stabilizer calculations for one fixed vertex

Status: EXPLORED. The following calculation is reproducible in Python and provides a concrete organization of the one-fixed-vertex model. Its complete transfer to the generated SAT branches and any distinct originality claim remain **PENDING**.

5.1 Binary support incidence and a corrected variance calculation

For the known involution case $f = 1$, the rooted neighborhood has seven matched pairs and the second subconstituent has 42 transposed pairs. The outer vertices’ coordinates as nonmatched neighborhood pairs are already used in prior work, including Section 2 of Cesarz–Woldar [2].

Define C as the binary support-incidence matrix between the seven neighborhood pairs and the 42 outer orbits. Its entries indicate whether a neighborhood pair belongs to an outer orbit’s support; they are not cardinalities of intersections

with the union of both vertices' neighborhoods. The script `scripts/classify_z2_f1_incidence_matrix.py` constructs $C = [C_1 \mid C_1]$, where C_1 is the vertex-edge incidence matrix of K_7 , and checks

$$CC^T = 10I_7 + 2J_7. \quad (5.1)$$

For any binary 7×42 matrix satisfying (5.1), let k_j be the column sums. Then

$$\sum_j k_j = 84, \quad (5.2)$$

$$\sum_j k_j^2 = \mathbf{1}^T CC^T \mathbf{1} = 10 \cdot 7 + 2 \cdot 7^2 = 168, \quad (5.3)$$

$$\sum_j (k_j - 2)^2 = 168 - 4 \cdot 84 + 4 \cdot 42 = 0. \quad (5.4)$$

Thus each column has weight 2, and every unordered pair of rows occurs in exactly two columns. This is a conditional incidence calculation, not a proof of the graph-to-encoding interface.

The previous manuscript incorrectly identified $\sum_j k_j^2$ with $\text{Tr}(C^T C)$. The latter equals $\text{Tr}(CC^T) = 84$, not 168. The correct expression is the quadratic form $\mathbf{1}^T CC^T \mathbf{1}$, also used in the script's explanatory derivation.

5.2 Two relabeling groups, not one

The undecorated binary incidence matrix allows independent swaps of the 21 pairs of duplicate columns. Its row/column relabeling group is $(S_2)^{21} \rtimes S_7$, of order 10,569,646,080. This must not be identified with the signed rooted coordinate group

$$W = (\mathbb{Z}_2)^7 \rtimes S_7, \quad |W| = 645,120. \quad (5.5)$$

The global sign flip lies in the kernel of the action of W on the 42 orbit labels. Its action on the full signed coordinate system and its induced action on orbit labels should not be conflated.

The audited Python calculation returns

```
transitive_on_orbits: True
pair_orbit_sizes: [21, 420, 420]
stab_00_order: 15360
stab_00_partition_sizes: [1, 1, 20, 20]
```

One singleton is the distinguished orbit itself. The remaining 41 labels partition into classes of sizes 1, 20, 20: identical support with opposite phase, intersecting support, and disjoint support. The branch construction chooses representatives of these classes for the proposed $K_{2,2}$ partner of the distinguished orbit.

The values 15,360 and 768 are stabilizer orders in this description, not measured speedup factors. A complete proof that the graph reduction, partner requirement, all symmetry cuts, and CNF instances preserve every admissible graph is not supplied by reproducing these group calculations. The mathematical justification and implementation must be audited together.

6 Inconclusive search, reproducibility, and remaining obligations

6.1 EXPLORED does not mean probabilistically close to a proof

Historical reports record substantial solver activity, but conflict totals across seeds need not correspond to disjoint regions of the search space. Variable elimination, clause reduction, and proof-file size do not supply a calibrated fraction of the mathematical problem solved or a completion forecast.

Earlier numerical bounds on satisfiability probability and tables of projected UNSAT completion probabilities are withdrawn. No justified prior, likelihood model, calibration set, or independence argument was established for these instances. The rigidity conjecture does not provide those missing statistical assumptions.

Remote process counts, machine load, disk capacity, and uptime were not checked in this audit. Older operational figures are historical reports, not live state. A stopped process, timeout, or incomplete proof trace remains **EXPLORED**; preserving a DRAT prefix does not show that a fresh process can resume the prior learned solver state.

6.2 Reproduction and validation requirements

The audit ran the following commands from the repository root:

```
lake build
python3 -m unittest discover -s tests -p test_canonical_sat_compilers.py -v
```

The initial results were a successful 32-job build with three sorry warnings and 13 passing tests; the subsequent implementation follow-up passes the 39 selected tests described above. Targeted axiom checks and the small counterexample establish the more limited interpretation described above. The README supplies the axiom-audit commands and commands for independently checking the retained $f = 3$ formula/proof pairs. Those DRAT checks were not rerun for this revision.

A reproducible future experiment should retain the generator revision and options, exact CNF and hash, solver version and command, resource limits, full terminal log, proof or decoded candidate, and independent verification output. Encoding validity and branch coverage are additional mathematical obligations. Merely matching descriptive filenames is insufficient.

Historical audit entries are preserved with a superseding correction notice. The initial revision changed documentation only; the subsequent authorized implementation follow-up repaired encoders and branch indices, generated new inputs, and started one bounded local run of corrected C. Historical inputs and proofs were preserved, and no Lean proof or remote process was changed. Run-specific manifests, source snapshots, and status records are identified in the README.

7 Conclusion

The repository contains useful checked components and reproducible calculations, but its present value must be separated from mathematical novelty and end-to-end certification. The candidate checker and selected structural or arithmetic statements are **PROVED** within their audited scopes. The library with unfinished declarations and the limited tested encoders are **COMPILED**. Incidence calculations, uncertified analyses, and inconclusive searches are **EXPLORED**. Repairs, complete graph-to-encoding validation, missing formal interfaces, originality, and unresolved mathematical cases are **PENDING**.

The reduction of involutions to one fixed vertex and the exclusion of order 7 are antecedents, not discoveries of this work. A validated implementation, formalization, or controlled computational method could support a distinct contribution without another UNSAT result, but that contribution is not established by accumulated conflicts or by the existence of proof files.

Finally, every search imposing a nontrivial automorphism omits rigid graphs. Even a complete proof that $\text{Aut}(G) = \{1\}$ would leave the existence problem unresolved. No claim of a solution, a new graph-theoretic exclusion, or publication readiness is made.

Declarations

Statement of AI use. The repository and this revision were developed with AI assistance for code, mathematical exposition, and documentation review. Such assistance is not independent mathematical review. The earlier blanket assertion that every mathematical claim had been deterministically verified is withdrawn. Verification is limited to the exact propositions, test executions, and recorded formula-level checks described in this note.

Disclosure statement. No potential conflict of interest was reported by the author.

Acknowledgments. The author thanks the mathematical and open-source communities behind CaDiCaL, drat-trim, PySAT, and Lean 4.

Funding. No external funding was received for this study.

References

- [1] John H. Conway. Five \$1,000 problems. Proceedings of the OEIS50–DIMACS Conference, OEIS Foundation, 2014. Updated 2017.

- [2] Patrick G. Cesarz and Andrew J. Woldar. On the automorphism group of a putative Conway 99-graph. *Algebraic Combinatorics*, 8(2):379–398, 2025. doi: 10.5802/alco.418.
- [3] Andries E. Brouwer and Willem H. Haemers. *Spectra of Graphs*. Universitext. Springer, 2012. URL <https://homepages.cwi.nl/~aeb/math/ipm/>.
- [4] A. A. Makhnev. Symmetric graphs and their automorphisms. Lecture slides, Berlin, September 2009, 2009. URL <https://www.math.uni-bielefeld.de/~baumeist/sommerschule/makhnev.pdf>. Theorem 1 attributes the prime-order fixed-point classification to Makhnev and Minakova.
- [5] A. A. Makhnev and I. M. Minakova. On automorphisms of strongly regular graphs with the parameters $\lambda = 1$ and $\mu = 2$. *Discrete Mathematics and Applications*, 14(2):201–210, 2004. doi: 10.1515/156939204872374.
- [6] Majid Behbahani and Clement W. H. Lam. Strongly regular graphs with non-trivial automorphisms. *Discrete Mathematics*, 311(2–3):132–144, 2011. doi: 10.1016/j.disc.2010.10.005.
- [7] Dean Crnković and Marija Maksimović. Construction of strongly regular graphs having an automorphism group of composite order. *Contributions to Discrete Mathematics*, 15(1):22–41, 2020. doi: 10.55016/ojs/cdm.v15i1.62323.
- [8] Aalok Thakkar. A forced-structure reduction and verifiable bounds for Conway’s 99-graph. *arXiv preprint arXiv:2608.11211*, 2026.
- [9] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021.
- [10] Nathan Wetzler, Marijn J. H. Heule, and Warren A. Hunt, Jr. DRAT-trim: Efficient checking and trimming using expressive clausal proofs. In *Theory and Applications of Satisfiability Testing – SAT 2014*, Lecture Notes in Computer Science. Springer, 2014. doi: 10.1007/978-3-319-09284-3_31.